

# ModelSpec Introspector

User Guide & Technical Reference

*Automated Discovery & Documentation  
for vLLM Inference Deployments*



**Version** 1.0.0  
**License** Apache-2.0

## Contents

|          |                                           |          |
|----------|-------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                       | <b>2</b> |
| 1.1      | What It Does . . . . .                    | 2        |
| 1.2      | Architecture Overview . . . . .           | 2        |
| <b>2</b> | <b>Getting Started</b>                    | <b>2</b> |
| 2.1      | Prerequisites . . . . .                   | 2        |
| 2.2      | Installation . . . . .                    | 2        |
| 2.3      | Quick Start . . . . .                     | 3        |
| <b>3</b> | <b>Features</b>                           | <b>3</b> |
| 3.1      | Intelligent Framework Detection . . . . . | 3        |
| 3.2      | GPU Metrics Collection . . . . .          | 3        |
| 3.3      | Runtime Metrics via vLLM API . . . . .    | 4        |
| 3.4      | Multi-Pod Aggregation . . . . .           | 4        |
| 3.5      | Confidence Scoring . . . . .              | 4        |
| 3.6      | Execution Modes . . . . .                 | 5        |
| <b>4</b> | <b>Command Reference</b>                  | <b>5</b> |
| 4.1      | Scan Command . . . . .                    | 5        |
| 4.2      | Options . . . . .                         | 5        |
| <b>5</b> | <b>Output Formats</b>                     | <b>5</b> |
| 5.1      | ModelSpec YAML . . . . .                  | 5        |
| 5.2      | PIQC Facts Bundle . . . . .               | 6        |
| 5.3      | Facts Reference . . . . .                 | 6        |
| <b>6</b> | <b>Data Collection Flow</b>               | <b>7</b> |
| <b>7</b> | <b>RBAC Configuration</b>                 | <b>7</b> |
| <b>8</b> | <b>Troubleshooting</b>                    | <b>7</b> |
| <b>A</b> | <b>Complete CLI Help</b>                  | <b>8</b> |
| <b>B</b> | <b>Example Commands</b>                   | <b>8</b> |

# 1 Introduction

ModelSpec Introspector automatically discovers vLLM inference deployments running in Kubernetes clusters and generates comprehensive documentation about their configuration, resource utilization, and runtime performance.

## 1.1 What It Does

- Scans Kubernetes namespaces to identify vLLM workloads
- Collects GPU metrics directly from running pods via `nvidia-smi`
- Queries vLLM's OpenAI-compatible API for runtime statistics
- Generates standardized ModelSpec documentation in multiple formats
- Produces PIQC facts bundles with full data provenance

## 1.2 Architecture Overview

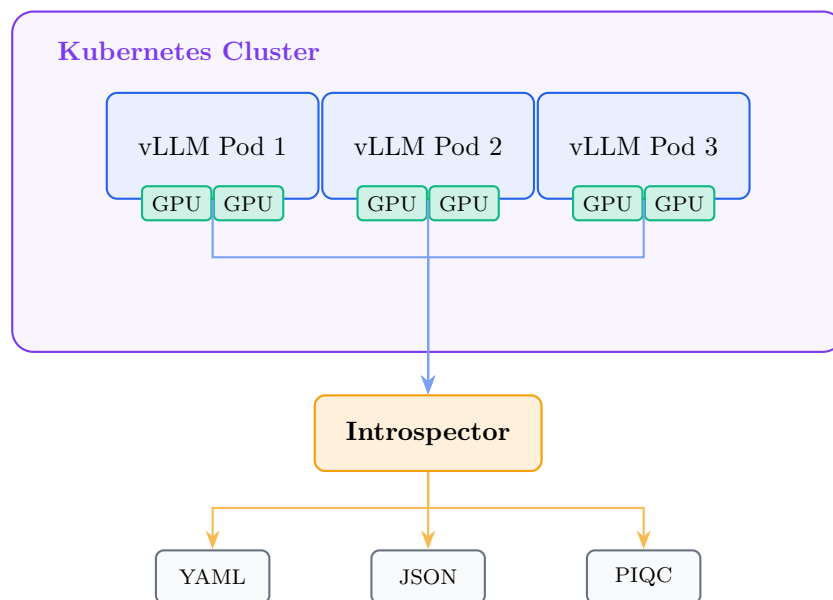


Figure 1: System Architecture

# 2 Getting Started

## 2.1 Prerequisites

- Python 3.10 or higher
- `kubectl` configured with cluster access
- RBAC permissions for pod listing, exec, and log access

## 2.2 Installation

```
pip install modelspec-introspector

# Verify installation
modelspec-introspector --version
```

## 2.3 Quick Start

```
# Scan all namespaces
modelspec-introspector scan

# Scan specific namespace with JSON output
modelspec-introspector scan --namespace production --output-format json

# Full scan with runtime metrics
modelspec-introspector scan --collect-runtime --output-piqc --verbose
```

## 3 Features

### 3.1 Intelligent Framework Detection

The introspector identifies vLLM deployments using a weighted multi-signal approach that analyzes container images, environment variables, command-line arguments, and Kubernetes labels.

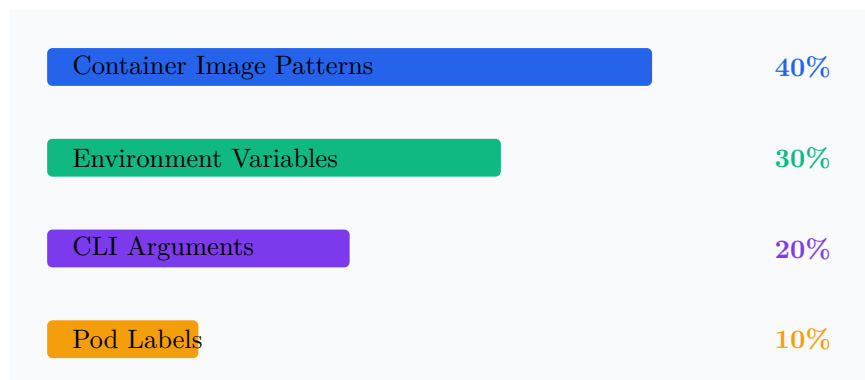


Figure 2: Detection Signal Weights

#### Recognized Patterns:

- `vllm/vllm-openai:*`
- `-tensor-parallel-size`
- `-max-model-len`
- `MODEL_NAME` env var
- `TENSOR_PARALLEL_SIZE`
- `app.kubernetes.io/name: vllm`

### 3.2 GPU Metrics Collection

Real-time GPU metrics are collected by executing `nvidia-smi` inside running pods:

| Metric       | Unit | Description                            |
|--------------|------|----------------------------------------|
| GPU Model    | —    | Hardware model name (A100, H100, etc.) |
| Memory Total | GB   | Total VRAM capacity                    |
| Memory Used  | GB   | Currently allocated memory             |
| Utilization  | %    | Compute core utilization               |
| Temperature  | °C   | Current GPU temperature                |
| Power Draw   | W    | Current power consumption              |

### 3.3 Runtime Metrics via vLLM API

When enabled with `-collect-runtime`, the introspector queries vLLM's built-in endpoints:

| Endpoint       | Data Collected                                  |
|----------------|-------------------------------------------------|
| GET /health    | Server availability status                      |
| GET /v1/models | Loaded model information                        |
| GET /metrics   | Prometheus metrics (throughput, latency, cache) |

#### Metrics Include:

- Requests running, waiting, and swapped
- Prompt and generation tokens per second
- KV cache utilization percentage
- Time-to-first-token (TTFT) percentiles
- End-to-end request latency percentiles

### 3.4 Multi-Pod Aggregation

For deployments with multiple replicas, metrics are automatically aggregated with statistical summaries:

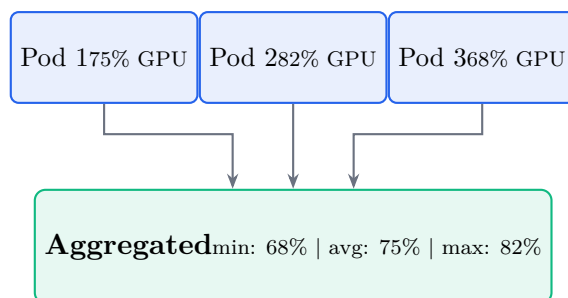


Figure 3: Multi-Pod Metric Aggregation

Statistical measures computed: **min**, **max**, **average**, **standard deviation**

### 3.5 Confidence Scoring

Model identification includes confidence scores based on evidence quality:

| Confidence | Range       | Evidence Source                                       |
|------------|-------------|-------------------------------------------------------|
| High       | 0.95 – 1.00 | Explicit <code>MODEL_NAME</code> environment variable |
| High       | 0.85 – 0.94 | CLI <code>-model</code> argument                      |
| Medium     | 0.70 – 0.84 | Volume path inference                                 |
| Low        | 0.50 – 0.69 | Container image tag inference                         |

### 3.6 Execution Modes

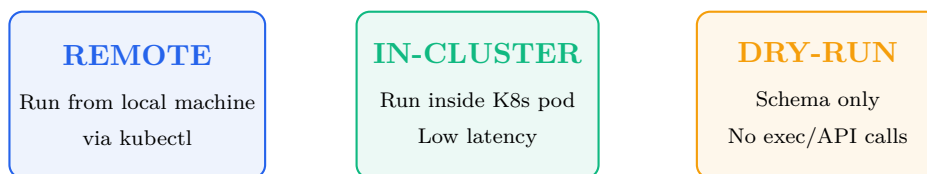


Figure 4: Execution Modes

## 4 Command Reference

### 4.1 Scan Command

```
modelspec-introspector scan [OPTIONS]
```

### 4.2 Options

| Option             | Default  | Description                 |
|--------------------|----------|-----------------------------|
| -kubeconfig PATH   | Auto     | Path to kubeconfig file     |
| -context NAME      | Current  | Kubernetes context          |
| -namespace NS      | All      | Namespace to scan           |
| -output-format FMT | table    | Output: table, yaml, json   |
| -output PATH       | ./output | Output directory            |
| -timeout SECS      | 30       | API timeout seconds         |
| -no-exec           | —        | Skip nvidia-smi collection  |
| -workers N         | 4        | Parallel worker threads     |
| -collect-runtime   | —        | Query vLLM API for metrics  |
| -aggregate         | Enabled  | Aggregate multi-pod metrics |
| -mode MODE         | auto     | remote, incluster, dry-run  |
| -output-piqc       | —        | Generate PIQC facts bundle  |
| -verbose, -v       | —        | Verbose logging             |
| -debug             | —        | Debug with stack traces     |

## 5 Output Formats

### 5.1 ModelSpec YAML

Standard output format for human-readable documentation:

```
apiVersion: modelspec.paralleliq.cloud/v1alpha1
kind: ModelSpec
metadata:
  name: llama-7b-vllm
  namespace: production
model:
  name: meta-llama/Llama-2-7b-hf
  architecture: llama
  parameters: "7B"
  confidence: 0.95
engine:
```

```

name: vllm
detectionConfidence: 0.92
inference:
  precision: bfloat16
  maxModelLen: 4096
  tensorParallelSize: 4
resources:
  replicas: 2
  gpus:
    - type: A100-SXM4-80GB
      memoryTotal: 80GB
      utilization: 75

```

## 5.2 PIQC Facts Bundle

Machine-readable format with full provenance tracking (piqc-facts.json):

```

{
  "schemaVersion": "piqc-scan.v0.1",
  "generatedAt": "2026-01-03T22:00:00Z",
  "tool": {"name": "modelspec-introspector", "version": "1.0.0"},
  "objects": [{
    "workloadId": "ns/production/deploy/llama-7b",
    "facts": {
      "runtime.engineType": {
        "value": "vllm",
        "dataConfidence": "high",
        "source": {"type": "scanner", "method": "framework_detection"}
      },
      "hardware.gpuCount": {
        "value": 4,
        "dataConfidence": "high",
        "source": {"type": "k8s_api", "method": "pod_resources"}
      }
    }
  }],
  "factErrors": []
}

```

## 5.3 Facts Reference

| Fact Key                | Type    | Description                   |
|-------------------------|---------|-------------------------------|
| runtime.engineType      | string  | Inference framework (vllm)    |
| vllm.dtype              | string  | Data type (bfloat16, float16) |
| vllm.maxModelLen        | integer | Maximum context length        |
| vllm.tensorParallelSize | integer | TP degree                     |
| hardware.gpuType        | string  | GPU model name                |
| hardware.gpuCount       | integer | Total GPU count               |
| hardware.gpuMemoryGB    | number  | GPU VRAM in GB                |
| obs.gpu.utilAvgPct      | number  | Average GPU utilization       |
| obs.vllm.tokensPerSec   | number  | Generation throughput         |
| model.name              | string  | HuggingFace model ID          |

## 6 Data Collection Flow

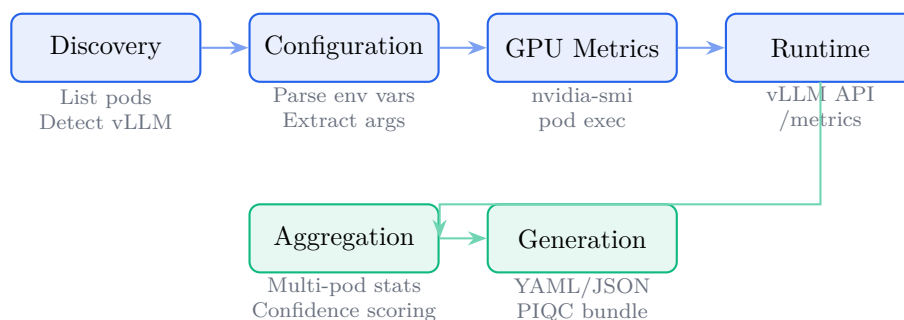


Figure 5: Data Collection Pipeline

## 7 RBAC Configuration

Required Kubernetes permissions:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: modelspec-introspector
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/exec", "pods/log"]
    verbs: ["get", "list", "create"]
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets"]
    verbs: ["get", "list"]
  - apiGroups: [""]
    resources: ["services"]
    verbs: ["get", "list"]
  
```

## 8 Troubleshooting

### GPU Type Shows “Unknown”

**Cause:** nvidia-smi not available in container

**Solution:** Use `-no-exec` flag, or ensure NVIDIA container runtime is configured

### Low Detection Confidence

**Cause:** Missing standard vLLM indicators

**Solution:** Add labels like `app.kubernetes.io/name: vllm` or use standard environment variables

### Permission Denied Errors

**Cause:** Insufficient RBAC permissions

**Solution:** Apply the ClusterRole and bind it to your service account



## A Complete CLI Help

```
Usage: modelspec-introspector scan [OPTIONS]

Scan Kubernetes cluster for vLLM model deployments.

Options:
--kubeconfig PATH          Path to kubeconfig file
--context TEXT             Kubernetes context to use
--namespace TEXT          Namespace to scan (default: all)
--output-format [table|yaml|json]
                           Output format (default: table)
--output PATH              Output directory (default: ./output)
--timeout INTEGER          API timeout seconds (default: 30)
--no-exec                  Skip pod exec for GPU metrics
--no-logs                  Skip log collection
--workers INTEGER          Parallel workers (default: 4)
-v, --verbose              Verbose output
--debug                    Debug mode with stack traces
--combined                 Single combined output file
--collect-runtime           Collect vLLM API runtime metrics
--aggregate / --no-aggregate
                           Aggregate multi-pod metrics (default: on)
--mode [auto|remote|incluster|dry-run]
                           Execution mode (default: auto)
--output-piqc              Generate piqc-facts.json
--help                     Show this message and exit
```

## B Example Commands

```
# Basic scan with table output
modelspec-introspector scan

# Production namespace with full metrics
modelspec-introspector scan \
  --namespace production \
  --collect-runtime \
  --output-format json \
  --output-piqc

# Dry-run for testing (no pod exec)
modelspec-introspector scan --mode dry-run --no-exec

# Debug mode with verbose logging
modelspec-introspector scan --debug --verbose
```